

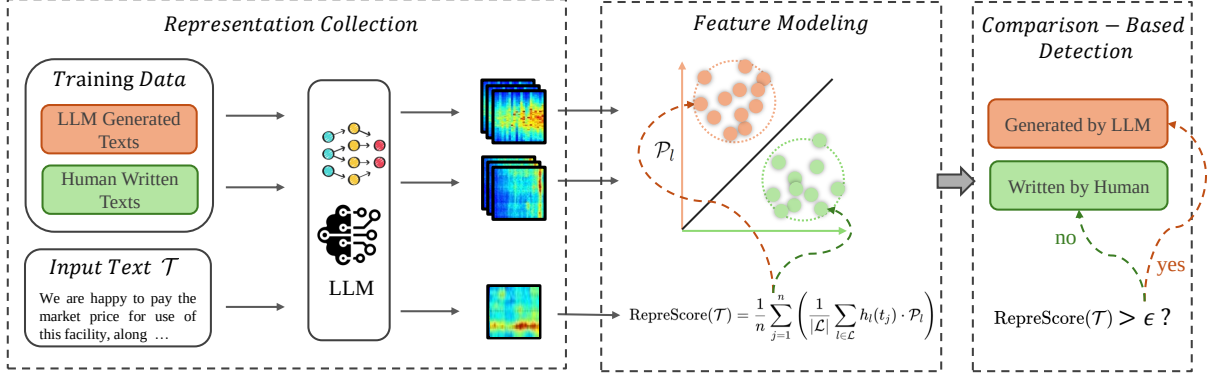


Figure 2: **The ReprGuard Framework Overview.** The framework processes text through a surrogate model to capture hidden representation patterns and collect representation of LGT and HWT. It employs Principal Component Analysis model distinct activation features, filtering out noise and identifying key features that distinguish LGT. Next, the framework calculates an overall projection score, which we called “ReprScore”, for the text to quantify how closely a text’s activation pattern aligns with LGT representation features. A threshold is then set based on the ReprScores from the training data. In the application tion, if the given text’s ReprScore exceeds this threshold, it is more likely to be generated by LLMs.

exceeds 20, their hidden representations begin to diverge substantially, with LGT demonstrating a consistently higher overall activation level compared to HWT. Moreover, at the same sequence positions, LGT and HWT remain relatively similar in the lower layers (approximately layers 1–11), while exhibiting more pronounced differences in layers 11–32.

3.2 Detecting by Representation Comparisons

Based on this hypothesis, we propose **ReprGuard** for detecting LGT using the hidden representation patterns of LLMs. By analyzing the activation patterns of a surrogate model during the processing of LGT and HWT, we aim to capture subtle but systematic differences in the activation patterns when LLMs process them, and extract the most significant representation feature for the effective detection of LGT. The overview framework of ReprGuard is as illustrated in Figure 2.

Representation Collection We first introduce a small training set, formalized as $\{(\mathcal{T}_{\text{LGT}}^i, \mathcal{T}_{\text{HWT}}^i) \mid i \in [1, N]\}$, where N is the number of LGT and HWT pairs in the dataset. We use a surrogate model $\mathcal{M}$ as an “observer” to collect the corresponding representation distribution when processing LGT and HWT, to capture the difference in their activation patterns. Specifically, for each text sequence $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$ containing n tokens, we use $\mathcal{M}$ with $\mathcal{L}$ layers to “observe” it. For each token t_j in $\mathcal{T}$, we collect the neural activa-

tion of the model $\mathcal{M}$ at each layer l , represented as $h_j^l \in \mathbb{R}^d$, where d is the dimension of the model’s hidden state. Based on this, the complete activation of the text $\mathcal{T}$ in model $\mathcal{M}$ can be formalized:

$$\mathcal{A}(\mathcal{T}) = \{h_j^l \mid j \in [1, n], l \in [1, \mathcal{L}]\} \quad (1)$$

For each text pair $(\mathcal{T}_{\text{LGT}}^i, \mathcal{T}_{\text{HWT}}^i)$, we capture the model’s representation activations from the end to the beginning of the token at all layers $\mathcal{L}$. We denote these activations as $\mathcal{A}(\mathcal{T}_{\text{LGT}}^i)$, and $\mathcal{A}(\mathcal{T}_{\text{HWT}}^i)$.

Feature Modeling The difference in activation patterns between LGT and HWT for each text pair can be denoted as $\Delta\mathcal{A}_i$, where:

$$\Delta\mathcal{A}_i = \mathcal{A}(\mathcal{T}_{\text{LGT}}^i) - \mathcal{A}(\mathcal{T}_{\text{HWT}}^i) \quad (2)$$

For each hidden layer l in $\mathcal{L}$, we capture this difference of neural activations from all text pairs:

$$\Delta\mathcal{A}^l = \{\Delta\mathcal{A}_i^l \mid i \in [1, N], l \in [1, \mathcal{L}]\} \quad (3)$$

where $\Delta\mathcal{A}_i^l = h_{\text{LGT}}^{l,i} - h_{\text{HWT}}^{l,i}$ represents the difference for the i -th text pair at layer l .

To filter out noise and identify key features distinguishing LGT, we perform Principal Component Analysis (PCA) on each $\Delta\mathcal{A}^l$:

$$\mathcal{P}_l = \text{PCA}(\Delta\mathcal{A}^l) \quad (4)$$

The resulting $\mathcal{P}_l$ represents the *probing vector* that differentiates between LGT and HWT at layer l , demonstrated that different types of text evoke

distinct neural activation pattern in the LLM. We project the activations of each token t_j in text $\mathcal{T}$ onto the probing vector across all layers $\mathcal{L}$, defining this as the **RepreScore**:

$$\text{RepreScore}(t_j) = \frac{1}{|\mathcal{L}|} \sum_{l \in \mathcal{L}} h_l(t_j) \cdot \mathcal{P}_l \quad (5)$$

The overall projection score for the text $\mathcal{T}$ is the mean of its tokens' RepreScores:

$$\text{RepreScore}(\mathcal{T}) = \frac{1}{n} \sum_{j=1}^n \text{RepreScore}(t_j) \quad (6)$$

Comparison-Based Detection Based on the calculated $\text{RepreScore}(\mathcal{T})$ for each sample in the training dataset, we determine the optimal threshold θ to balance the true positive rate (TPR) and the false positive rate (FPR):

$$\theta = \arg \max_{\theta'} (\text{TPR}(\theta') + (1 - \text{FPR}(\theta'))) \quad (7)$$

In the application phase, RepreScore measures how closely the activation pattern of the input text aligns with the unique neural features of LGT identified in the training dataset. For the detection result $\mathcal{S}(\mathcal{T})$, we calculate the $\text{RepreScore}(\mathcal{T})$ and compare it to the optimal threshold θ . If $\text{RepreScore}(\mathcal{T})$ exceeds θ , the input text is more likely to be generated by the LLM:

$$\mathcal{S}(\mathcal{T}) = \begin{cases} \text{LGT} & \text{if } \text{RepreScore}(\mathcal{T}) > \theta \\ \text{HWT} & \text{otherwise} \end{cases} \quad (8)$$

Effectiveness and Generalization Our *RepreGuard* framework effectively detects LGT by extracting features derived from more fundamental and comprehensive representation information. This demonstrates a stronger and more adaptable detection capability. We further validate this capability on LGT generated by four different LLMs and their corresponding HWT. The results in Figure 3 clearly show a distinct separation between the RepreScore distributions of LGT and HWT, with a consistent trend across different LLMs. Specifically, the RepreScore for HWT primarily ranges from -2 to 2. Despite variations in text distributions generated by different LLMs, the RepreScore for LGT consistently falls between 0 and 8. This indicates a universal threshold that can

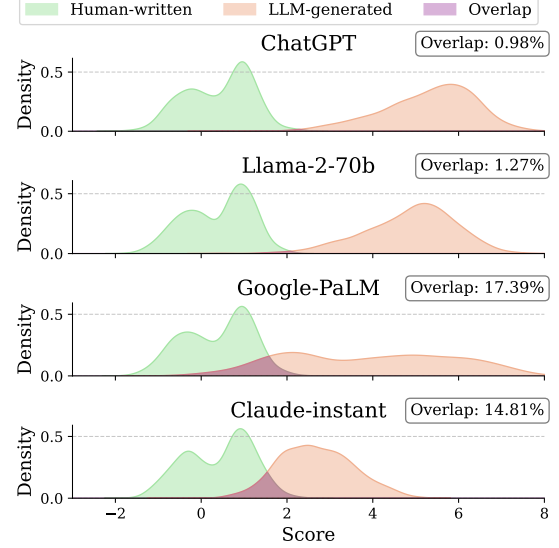


Figure 3: Comparison of RepreScores Distribution for Texts Generated by 4 Different LLMs (1000 Pairs LGT and HWT for Each LLM). The representation features used are modeling on Mutil-LLMs generated texts. The overlap means the overlap probability.

be effectively applied across text generated by various types of LLMs, eliminating the need for individual adjustments and highlighting RepreGuard's strong generalization capability.

4 Experiment

4.1 Experiment Setup

Dataset We utilized the DetectRL benchmark (Wu et al., 2024), a dataset specifically designed to evaluate the detection capabilities of HWT and LGT in scenarios closely aligned with real-world applications. The dataset comprises data from four domains that are more prone to misuse: academic writing (ArXiv Archive²), news writing (XSum (Narayan et al., 2018)), creative writing (Writing Prompts (Fan et al., 2018)), and social media texts (Yelp Review (Zhang et al., 2015)). For each domain, the dataset includes 2,800 pairs of LGT and HWT samples, with LGT generated using four widely used LLMs: ChatGPT (OpenAI, 2022), Claude-instant (Anthropic, 2023), Google-PaLM (Ghahramani, 2023), and Llama-2-70B.³

To ensure robust evaluation, we applied the bootstrapping method five times to the dataset of

²<https://www.kaggle.com/datasets/spsayakpaul/arxiv-paper-abstracts/data>

³<https://huggingface.co/meta-llama/Llama-2-70b-chat-hf>

each LLM, sampling a training set consisting of 512 pairs of samples and a test set consisting of 1,000 pairs of samples for each iteration. Additionally, to construct the multi-LLM dataset, we combined the data of four different LLMs and applied the bootstrap method five times equally on the training sets of different LLMs.

Baselines We compare RepreGuard with both fine-tuning-based and statistics-based methods to detect LGT:

- **RoBERTa-based classifier** (Liu et al., 2019): A supervised approach by fine-tuning pre-training language model (PLM) as a classifier. We use the RoBERTa-base consistent with the OpenAI detector (Solaiman et al., 2019) for training.
- **LRR** (Su et al., 2023): A statistics-based method based on the ratio of Log-likelihood and Log-rank. We use GPT-neo-2.7B for scoring following Bao et al. (2024) experiments.
- **DetectGPT** (Mitchell et al., 2023): A statistics-based zero-shot method based on the Log-probability curvature of the perturbed text. We use T5-small (Raffel et al., 2020) to perturb text, and use GPT-Neo-2.7B (Black et al., 2021) for scoring following Bao et al. (2024).
- **Fast-DetectGPT** (Fast-Detect.; Bao et al. 2024): A statistics-based zero-shot method that using a sampling strategy to replace the perturbation strategy of DetectGPT. We use GPT-Neo-2.7B (Black et al., 2021) for scoring and GPT-J-6B (Wang and Komatsuzaki, 2021) for sample generation following the best setting reported.
- **Straight-forward Detector** (Str-Detect.): A zero-shot approach that directly asks LLM for both HWT and LGT. This detector is for better comparison with our method using the intrinsic mechanism of LLMs.
- **Binoculars** (Hans et al., 2024): A statistics-based zero-shot method that uses a pair of LLMs to calculate the ratio of perplexity and cross-perplexity for detection. We use Falcon-7B and Falcon-7B-Instruct (Penedo et al., 2023) for detection following the best setting reported.

Metrics Following Mitchell et al. (2023), we utilized the AUROC to evaluate the performance of the detector as a binary classification model. In the LGT detection task, the most worrying harm

usually comes from false positives, that is, HWT is incorrectly labeled as LGT. Therefore, we also focus on the TPR at low FPR and follow the experimental setting of Hans et al. (2024) to adopt a standard FPR threshold of 0.01% TPR (TPR@0.01).

ID and OOD Detection Setting Our study examines both the performance of ID and in a strict zero-shot detection scenario, where we use a purely “OOD” setting to distinguish LGT from HWT, consistent with the “out-of-domain” claims in Binoculars (Hans et al., 2024) and Ghostbuster (Verma et al., 2024). In fact, many previous zero-shot works such as DetectGPT (Mitchell et al., 2023) are in different experimental settings for zero-shot detection, where they use the test set to define the threshold and get the best performance on the test set. We argue this experimental setting may limit the development of truly effective statistics-based zero-shot detectors. Therefore, our experimental setting is strictly aligned with the real-world scenario to ensure that the detector is robust and not overly dependent on any particular dataset. We use the training data to make the decision thresholds to detect text generated by unknown LLMs.

4.2 ID and OOD Performance

Our experiments evaluate both ID and OOD performance of the detectors, as shown in Table 1. The results demonstrate that RepreGuard achieves the best overall performance, excelling in both ID and OOD scenarios, achieving an average of $94.92_{\pm 0.70}\%$ and $82.44_{\pm 1.84}\%$ in AUROC and TPR@0.01, respectively.

ID Performance ID performance refers to the detectors’ ability to detect instances within the same distribution as the training dataset. The results show that RepreGuard outperforms other detectors in both AUROC and TPR@0.01 in ID setting. Specifically, it achieves $96.34_{\pm 0.27}\%$ AUROC and $83.74_{\pm 1.56}\%$ TPR@0.01, demonstrating its strength in detecting data from the same distribution as the training datasets. Binoculars is the second-best performing method in most settings, particularly in the AUROC metric, with an average AUROC of 92.16% while its TPR@0.01 is only achieving an average of 58.15%. Fine-tuning classifiers based on RoBERTa generally performs well to some extent, especially in terms of AUROC, with values rarely approaching RepreGuard (e.g., $98.38_{\pm 0.32}\%$ AUROC on ChatGPT